

PartnerAPI

Public API Programme for a Travel Booking Company

API Documentation

Document Purpose: Partner-facing reference for the standard PartnerAPI interface

Contents

1	Introduction	4
1.1	Purpose	4
1.2	API Scope	4
2	API Design Principles	4
3	Base API Information	5
3.1	Environments	5
3.2	Content Type	5
3.3	HTTP Methods	5
4	Authentication	6
4.1	Partner Authentication	6
4.2	Authentication Outcomes	6
5	Hotel Search	6
5.1	Endpoint	6
5.2	Example Request	7
5.3	Example Response	7
6	Availability	7
6.1	Purpose	7
6.2	Room Count	7

7	Create Booking	8
7.1	Endpoint	8
7.2	Example Request	8
7.3	Example Response	8
8	Retrieve Booking	8
8.1	Endpoint	8
8.2	Example Request	9
8.3	Example Response	9
9	Cancel Booking	9
9.1	Endpoint	9
9.2	Example Request	9
9.3	Example Response	9
10	API Key Management	9
11	Rate Limiting	10
11.1	Standard Rate Limit	10
11.2	Rate-Limit Response	10
12	Access Control	11
13	Error Handling	11
13.1	Error Response Structure	11
14	Sandbox	12
15	Usage Reporting	12
16	Security Requirements	13
17	Partner Onboarding	13
18	Standard API Endpoint Summary	13
19	Example End-to-End Workflow	14
20	API Quality and Operational Expectations	14

21 Versioning and Change Control	15
22 Standard Scope Boundary	16
23 Conclusion	16

1 Introduction

1.1 Purpose

PartnerAPI provides a standard application programming interface that allows approved partner travel agencies to search hotel availability and create and manage hotel bookings without logging into the travel company's main website.

The API is designed for partner agencies that require programmatic access to the company's hotel booking capabilities.

The initial programme targets five partner agencies for the first production release, with the remaining partners planned for subsequent onboarding.

1.2 API Scope

The standard API provides the following capabilities:

- Partner authentication.
- Hotel search.
- Hotel and room availability.
- Hotel booking creation.
- Booking retrieval.
- Booking cancellation.
- API key management.
- Sandbox access.
- Usage reporting.
- Rate limiting and access control.

The API is intended to provide a consistent interface for multiple partner agencies. Partner-specific functionality is not part of the standard API release and must be handled through formal change control.

2 API Design Principles

The PartnerAPI follows the following design principles:

1. **Standardised interface:** All partners use the same documented API contract.
2. **Secure access:** Requests require valid partner credentials and appropriate authorisation.

3. **Controlled usage:** Request rates are controlled to protect the platform and maintain service quality.
4. **Loose coupling:** The public API acts as an interface between partner systems and internal hotel/booking services.
5. **Clear error handling:** Invalid requests, authentication failures, access violations and rate-limit violations return explicit HTTP error responses.
6. **Sandbox support:** Partners can test integrations before production access.

3 Base API Information

3.1 Environments

The programme provides two logical environments:

Table 1: API Environments

Environment	Purpose	Access
Sandbox	Integration and partner testing before production use	Partner sandbox credentials
Production	Live hotel search and booking operations	Approved production partner credentials

The exact deployment hostname is environment-specific and should be provided to the partner during onboarding.

3.2 Content Type

Requests containing a body use JSON:

```
Content-Type: application/json
```

Successful responses are returned as JSON:

```
Content-Type: application/json
```

3.3 HTTP Methods

The API uses conventional HTTP methods:

Table 2: HTTP Methods

Method	Purpose
GET	Retrieve information or search for resources
POST	Create a new resource or submit an operation
DELETE	Remove or cancel a resource where supported

4 Authentication

4.1 Partner Authentication

Each approved partner is issued an API key.

The API key identifies the partner making the request and is used to control access to PartnerAPI resources.

The API key must be supplied with API requests using the authentication header defined during partner onboarding.

A representative request is:

```
Authorization: Bearer <PARTNER_API_KEY>
```

The actual production credential must never be embedded directly into publicly accessible source code or documentation examples.

4.2 Authentication Outcomes

Table 3: Authentication Outcomes

Condition	Expected result
Valid API key	Request proceeds to authorisation and rate-limit checks
Missing API key	Request is rejected
Invalid API key	Request is rejected
Revoked API key	Request is rejected

5 Hotel Search

5.1 Endpoint

```
GET /api/hotels/search
```

The hotel search operation allows a partner to request hotels matching supplied search criteria.

5.2 Example Request

```
GET /api/hotels/search?destination=Singapore&checkIn=2026-11-10&checkOut=2026-11-12&rooms=1
```

5.3 Example Response

```
{
  "results": [
    {
      "hotelId": "HTL001",
      "name": "Example Hotel",
      "rooms": [
        {
          "roomId": "RM001",
          "name": "Standard Room",
          "available": true
        }
      ]
    }
  ]
}
```

The example illustrates the response structure. Actual hotel inventory and commercial data are provided by the connected booking system.

6 Availability

6.1 Purpose

Availability information identifies whether suitable rooms can be booked for the requested search criteria.

Availability is returned as part of the hotel search and may include room-level availability information.

6.2 Room Count

The standard API supports a room-count range of:

$$1 \leq \text{rooms} \leq 10$$

The boundary values are important because they are also used in the testing equivalence-class and boundary-value analysis.

Table 4: Room Count Validation

Room Count	Result
0	Reject as invalid
1	Accept
2–9	Accept
10	Accept
11 or greater	Reject as invalid

7 Create Booking

7.1 Endpoint

```
POST /api/bookings
```

The create-booking operation submits a selected hotel and room choice for booking.

7.2 Example Request

```
{
  "hotelId": "HTL001",
  "roomId": "RM001",
  "checkIn": "2026-11-10",
  "checkOut": "2026-11-12",
  "rooms": 1
}
```

7.3 Example Response

```
{
  "bookingId": "BK10001",
  "status": "CONFIRMED",
  "hotelId": "HTL001",
  "roomId": "RM001"
}
```

The booking identifier returned by the API is subsequently used for retrieving or cancelling the booking.

8 Retrieve Booking

8.1 Endpoint


```
GET /api/bookings/{bookingId}
```

8.2 Example Request

```
GET /api/bookings/BK10001
```

8.3 Example Response

```
{
  "bookingId": "BK10001",
  "status": "CONFIRMED",
  "hotelId": "HTL001",
  "roomId": "RM001"
}
```

A partner can retrieve only bookings that the authenticated partner is authorised to access.

9 Cancel Booking

9.1 Endpoint

```
DELETE /api/bookings/{bookingId}
```

9.2 Example Request

```
DELETE /api/bookings/BK10001
```

9.3 Example Response

```
{
  "bookingId": "BK10001",
  "status": "CANCELLED"
}
```

Cancellation is subject to the booking rules enforced by the underlying booking system.

10 API Key Management

API keys provide partner identification and access control.

The platform supports API key management for approved partners.

Typical administrative operations include:

- issuing a partner API key;
- validating an API key;
- revoking an API key; and
- replacing credentials when required.

A revoked API key must no longer provide access to protected API operations.

API key management is controlled by authorised platform administrators rather than ordinary booking requests.

11 Rate Limiting

11.1 Standard Rate Limit

The standard request-rate policy is:

100 requests per minute

The boundary behaviour is defined as follows:

Table 5: Request Rate Boundary Behaviour

Request Rate	Expected Result
99 requests/minute	Accepted
100 requests/minute	Accepted according to policy
101 requests/minute	Rejected with HTTP 429

11.2 Rate-Limit Response

When a partner exceeds the configured request rate, the API returns:

HTTP/1.1 429 Too Many Requests

Example:

```
{
  "error": "RATE_LIMIT_EXCEEDED",
  "message": "Request rate limit exceeded."
}
```

Rate limiting protects the platform from excessive request volumes and ensures fair access across partners.

12 Access Control

PartnerAPI uses partner-level authorisation.

A valid API key identifies the partner, but access to a resource must also satisfy the applicable authorisation rules.

For example, a partner must not retrieve or modify a booking belonging to another partner.

The access-control decision can be represented as:

Table 6: Partner Access Decision

Authorised	API Key	Rate Limit	Result
No	Any	Any	Reject
Yes	Invalid	Any	Reject
Yes	Valid	Exceeded	Reject with HTTP 429
Yes	Valid	Within limit	Allow request

13 Error Handling

PartnerAPI uses HTTP status codes to communicate the broad result of a request.

Table 7: Standard HTTP Error Responses

Code	Meaning	Example condition
400	Bad Request	Invalid request parameters
401	Unauthorized	Missing or invalid authentication
403	Forbidden	Authenticated partner is not authorised for the resource
404	Not Found	Requested hotel or booking cannot be found
409	Conflict	Request conflicts with current booking state
429	Too Many Requests	Partner exceeded rate limit
500	Internal Server Error	Unexpected server-side failure

13.1 Error Response Structure

A consistent error structure should be used where applicable:

```
{
  "error": "ERROR_CODE",
  "message": "Human-readable description of the error."
}
```

Error messages should provide enough information for a partner to correct the request without exposing internal implementation details.

14 Sandbox

The sandbox provides a controlled environment for partner integration testing before production access.

Partners should use the sandbox to validate:

- authentication;
- request construction;
- hotel search;
- booking workflows;
- cancellation;
- error handling;
- rate-limit behaviour; and
- integration logic.

Sandbox credentials are separate from production credentials.

A successful sandbox integration does not automatically grant production access; production onboarding remains subject to the partner approval process.

15 Usage Reporting

PartnerAPI records API usage to support operational monitoring and partner management.

Usage reporting may include information such as:

- partner identity;
- request type;
- request count;
- response status;
- rate-limit events; and
- usage period.

The purpose of usage reporting is to provide visibility into API consumption and support management of partner access.

16 Security Requirements

The API must enforce the following security principles:

1. API credentials must be protected from unauthorised disclosure.
2. Requests to protected resources must be authenticated.
3. Partner access must be authorised at the resource level.
4. Revoked credentials must not continue to provide access.
5. Rate limiting must protect the service from excessive request volume.
6. Error responses must not expose unnecessary internal implementation details.

The API must be deployed using secure transport in production environments.

17 Partner Onboarding

The standard onboarding process is:

1. Partner is approved for API access.
2. Partner receives sandbox credentials.
3. Partner reviews the API documentation.
4. Partner integrates against the sandbox.
5. Partner validates authentication, search, booking and cancellation.
6. Partner completes required testing.
7. Production credentials are issued following approval.

The initial programme plans to onboard five partners during the first production release.

18 Standard API Endpoint Summary

Table 8: PartnerAPI Endpoint Summary

Method	Endpoint	Purpose
GET	/api/hotels/search	Search available hotels using supplied criteria.

Method	Endpoint	Purpose
POST	/api/bookings	Create a hotel booking.
GET	/api/bookings/{bookingId}	Retrieve an existing booking.
DELETE	/api/bookings/{bookingId}	Cancel an existing booking where permitted.

19 Example End-to-End Workflow

A normal partner booking flow is:

1. Partner sends an authenticated hotel-search request.
2. API validates the API key.
3. API checks partner authorisation.
4. API checks the partner's request-rate limit.
5. API forwards the search request to the relevant hotel/booking service.
6. Available hotels and rooms are returned.
7. Partner selects a suitable room.
8. Partner submits a booking request.
9. API validates authentication, authorisation and rate-limit conditions.
10. Booking service processes the booking.
11. API returns the booking identifier and status.
12. Partner can retrieve the booking later using its booking identifier.
13. Partner can request cancellation when permitted.

This workflow reflects the intended separation between the public PartnerAPI interface and the internal booking services.

20 API Quality and Operational Expectations

The API documentation and implementation should support the project's defined quality objectives.

The project testing strategy specifically validates:

- valid and invalid authentication;

- hotel search;
- booking creation;
- booking retrieval;
- booking cancellation;
- API key revocation;
- rate-limit behaviour;
- unauthorised partner access;
- sandbox operation; and
- usage reporting.

The defined request-rate boundary is tested at 99, 100 and 101 requests per minute.

The API quality target also includes an API error-rate threshold below:

0.5%

as defined by the project's quality gates.

21 Versioning and Change Control

PartnerAPI is intended to provide a stable standard interface for multiple partner agencies.

Changes to the standard API must therefore be controlled.

A proposed change should be:

1. captured;
2. understood in terms of its reason;
3. analysed for schedule, cost, scope and quality impact;
4. formally decided;
5. incorporated into the project plan when approved; and
6. implemented and verified.

Partner-specific requirements that are not part of the standard API should not be added informally.

They must follow the project's formal change-control process.

Table 9: API Scope Boundary

In Scope	Out of Scope
Partner authentication	Partner-specific custom functionality without change approval
Hotel search	One-off workflows designed only for a single partner
Hotel availability	Unapproved changes to the standard API contract
Booking creation	Features that cannot be justified for the common partner platform
Booking retrieval	Custom functionality added without impact analysis
Booking cancellation	
Sandbox access	
API key management	
Usage reporting	
Rate limiting and access control	

22 Standard Scope Boundary

This scope boundary protects the first release from uncontrolled expansion while still allowing future enhancements through formal change management.

23 Conclusion

PartnerAPI provides a standard interface through which approved travel agencies can programmatically access hotel search and booking capabilities.

The API documentation defines the core partner workflow, authentication, authorisation, rate limiting, hotel search, booking operations, cancellation, sandbox usage and reporting.

The interface is deliberately standardised so that the first release can serve multiple partners without introducing unnecessary partner-specific complexity.

Any requirement outside this standard scope should be evaluated through formal change control before implementation.